

that $\gcd(ab, p) = p$. We also have that $\gcd(a, p) = 1$ and $\gcd(b, p) = 1$, since the only divisors of p are 1 and p , and we assumed that p divides neither a nor b . Theorem 31.6 then implies that $\gcd(ab, p) = 1$, contradicting $\gcd(ab, p) = p$. This contradiction completes the proof. ■

A consequence of Theorem 31.7 is that any composite integer can be uniquely factored into a product of primes. Exercise 31.1-11 asks you to provide a proof.

Theorem 31.8 (Unique prime factorization)

There is exactly one way to write any composite integer a as a product of the form

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r},$$

where the p_i are prime, $p_1 < p_2 < \cdots < p_r$, and the e_i are positive integers. ■

As an example, the unique prime factorization of the number 6000 is $2^4 \cdot 3^1 \cdot 5^3$.

Exercises

31.1-1

Prove that if $a > b > 0$ and $c = a + b$, then $c \bmod a = b$.

31.1-2

Prove that there are infinitely many primes. (*Hint:* Show that none of the primes $p_1, p_2, \dots, p_k$ divide $(p_1 p_2 \cdots p_k) + 1$.)

31.1-3

Prove that if $a \mid b$ and $b \mid c$, then $a \mid c$.

31.1-4

Prove that if p is prime and $0 < k < p$, then $\gcd(k, p) = 1$.

31.1-5

Prove Corollary 31.5.

31.1-6

Prove that if p is prime and $0 < k < p$, then $p \mid \binom{p}{k}$. Conclude that for all integers a and b and all primes p ,

$$(a + b)^p = a^p + b^p \pmod{p}.$$

31.1-7

Prove that if a and b are any positive integers such that $a \mid b$, then

$$(x \bmod b) \bmod a = x \bmod a$$

for any x . Prove, under the same assumptions, that

$$x = y \pmod{b} \text{ implies } x = y \pmod{a}$$

for any integers x and y .

31.1-8

For any integer $k > 0$, an integer n is a *k th power* if there exists an integer a such that $a^k = n$. Furthermore, $n > 1$ is a *nontrivial power* if it is a k th power for some integer $k > 1$. Show how to determine whether a given β -bit integer n is a nontrivial power in time polynomial in β .

31.1-9

Prove equations (31.6)–(31.10).

31.1-10

Show that the gcd operator is associative. That is, prove that for all integers a , b , and c , we have

$$\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c) .$$

★ 31.1-11

Prove Theorem 31.8.

31.1-12

Give efficient algorithms for the operations of dividing a β -bit integer by a shorter integer and of taking the remainder of a β -bit integer when divided by a shorter integer. Your algorithms should run in $\Theta(\beta^2)$ time.

31.1-13

Give an efficient algorithm to convert a given β -bit (binary) integer to a decimal representation. Argue that if multiplication or division of integers whose length is at most β takes $M(\beta)$ time, where $M(\beta) = \Omega(\beta)$, then you can convert binary to decimal in $O(M(\beta) \lg \beta)$ time. (*Hint:* Use a divide-and-conquer approach, obtaining the top and bottom halves of the result with separate recursions.)

31.1-14

Professor Marshall sets up n lightbulbs in a row. The lightbulbs all have switches, so that if he presses a bulb, it toggles on if it was off and off if it was on. The lightbulbs all start off. For $i = 1, 2, 3, \dots, n$, the professor presses bulb $i, 2i, 3i, \dots$. After the last press, which lightbulbs are on? Prove your answer.

31.2 Greatest common divisor

In this section, we describe Euclid's algorithm for efficiently computing the greatest common divisor of two integers. When we analyze the running time, we'll see a surprising connection with the Fibonacci numbers, which yield a worst-case input for Euclid's algorithm.

We restrict ourselves in this section to nonnegative integers. This restriction is justified by equation (31.8), which states that $\gcd(a, b) = \gcd(|a|, |b|)$.

In principle, for positive integers a and b , their prime factorizations suffice to compute $\gcd(a, b)$. Indeed, if

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}, \quad (31.11)$$

$$b = p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r}, \quad (31.12)$$

with 0 exponents being used to make the set of primes $p_1, p_2, \dots, p_r$ the same for both a and b , then, as Exercise 31.2-1 asks you to show,

$$\gcd(a, b) = p_1^{\min\{e_1, f_1\}} p_2^{\min\{e_2, f_2\}} \cdots p_r^{\min\{e_r, f_r\}}. \quad (31.13)$$

The best algorithms to date for factoring do not run in polynomial time. Thus, this approach to computing greatest common divisors seems unlikely to yield an efficient algorithm.

Euclid's algorithm for computing greatest common divisors relies on the following theorem.

Theorem 31.9 (GCD recursion theorem)

For any nonnegative integer a and any positive integer b ,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Proof We will show that $\gcd(a, b)$ and $\gcd(b, a \bmod b)$ divide each other. Since they are both nonnegative, equation (31.5) then implies that they must be equal.

We first show that $\gcd(a, b) \mid \gcd(b, a \bmod b)$. If we let $d = \gcd(a, b)$, then $d \mid a$ and $d \mid b$. By equation (3.11) on page 64, $a \bmod b = a - qb$, where $q = \lfloor a/b \rfloor$. Since $a \bmod b$ is thus a linear combination of a and b , equation (31.4) implies that $d \mid (a \bmod b)$. Therefore, since $d \mid b$ and $d \mid (a \bmod b)$, Corollary 31.3 implies that $d \mid \gcd(b, a \bmod b)$, that is,

$$\gcd(a, b) \mid \gcd(b, a \bmod b). \quad (31.14)$$

Showing that $\gcd(b, a \bmod b) \mid \gcd(a, b)$ is almost the same. If we now let $d = \gcd(b, a \bmod b)$, then $d \mid b$ and $d \mid (a \bmod b)$. Since $a = qb + (a \bmod b)$,

where $q = \lfloor a/b \rfloor$, we have that a is a linear combination of b and $(a \bmod b)$. By equation (31.4), we conclude that $d \mid a$. Since $d \mid b$ and $d \mid a$, we have that $d \mid \gcd(a, b)$ by Corollary 31.3, so that

$$\gcd(b, a \bmod b) \mid \gcd(a, b) . \quad (31.15)$$

Using equation (31.5) to combine equations (31.14) and (31.15) completes the proof. ■

Euclid's algorithm

Euclid's *Elements* (circa 300 B.C.E.) describes the following gcd algorithm, although its origin might be even earlier. The recursive procedure EUCLID implements Euclid's algorithm, based directly on Theorem 31.9. The inputs a and b are arbitrary nonnegative integers.

```

EUCLID( $a, b$ )
1  if  $b == 0$ 
2      return  $a$ 
3  else return EUCLID( $b, a \bmod b$ )

```

For example, here is how the procedure computes $\gcd(30, 21)$:

$$\begin{aligned}
 \text{EUCLID}(30, 21) &= \text{EUCLID}(21, 9) \\
 &= \text{EUCLID}(9, 3) \\
 &= \text{EUCLID}(3, 0) \\
 &= 3 .
 \end{aligned}$$

This computation calls EUCLID recursively three times.

The correctness of EUCLID follows from Theorem 31.9 and the property that if the algorithm returns a in line 2, then $b = 0$, so that by equation (31.9), $\gcd(a, b) = \gcd(a, 0) = a$. The algorithm cannot recurse indefinitely, since the second argument strictly decreases in each recursive call and is always nonnegative. Therefore, EUCLID always terminates with the correct answer.

The running time of Euclid's algorithm

Let's analyze the worst-case running time of EUCLID as a function of the size of a and b . The overall running time of EUCLID is proportional to the number of recursive calls it makes. The analysis assumes that $a > b \geq 0$, that is, the first argument is greater than the second argument. Why? If $b = a > 0$, then $a \bmod b = 0$ and the procedure terminates after one recursive call. If $b > a \geq 0$,

then the procedure makes just one more recursive call than when $a > b$, because in this case $\text{EUCLID}(a, b)$ immediately makes the recursive call $\text{EUCLID}(b, a)$, and now the first argument is greater than the second.

Our analysis relies on the Fibonacci numbers F_k , defined by the recurrence equation (3.31) on page 69.

Lemma 31.10

If $a > b \geq 1$ and the call $\text{EUCLID}(a, b)$ performs $k \geq 1$ recursive calls, then $a \geq F_{k+2}$ and $b \geq F_{k+1}$.

Proof The proof proceeds by induction on k . For the base case of the induction, let $k = 1$. Then, $b \geq 1 = F_2$, and since $a > b$, we must have $a \geq 2 = F_3$. Since $b > (a \bmod b)$, in each recursive call the first argument is strictly larger than the second. The assumption that $a > b$ therefore holds for each recursive call.

Assuming inductively that the lemma holds if the procedure makes $k - 1$ recursive calls, we shall prove that the lemma holds for k recursive calls. Since $k > 0$, we have $b > 0$, and $\text{EUCLID}(a, b)$ calls $\text{EUCLID}(b, a \bmod b)$ recursively, which in turn makes $k - 1$ recursive calls. The inductive hypothesis then implies that $b \geq F_{k+1}$ (thus proving part of the lemma), and $a \bmod b \geq F_k$. We have

$$\begin{aligned} b + (a \bmod b) &= b + (a - b \lfloor a/b \rfloor) && \text{(by equation (3.11))} \\ &\leq a, \end{aligned}$$

since $a > b > 0$ implies $\lfloor a/b \rfloor \geq 1$. Thus,

$$\begin{aligned} a &\geq b + (a \bmod b) \\ &\geq F_{k+1} + F_k \\ &= F_{k+2}. \end{aligned}$$

■

The following theorem is an immediate corollary of this lemma.

Theorem 31.11 (Lamé's theorem)

For any integer $k \geq 1$, if $a > b \geq 1$ and $b < F_{k+1}$, then the call $\text{EUCLID}(a, b)$ makes fewer than k recursive calls. ■

To show that the upper bound of Theorem 31.11 is the best possible, we'll show that the call $\text{EUCLID}(F_{k+1}, F_k)$ makes exactly $k - 1$ recursive calls when $k \geq 2$. We use induction on k . For the base case, $k = 2$, and the call $\text{EUCLID}(F_3, F_2)$ makes exactly one recursive call, to $\text{EUCLID}(1, 0)$. (We have to start at $k = 2$, because when $k = 1$ we do not have $F_2 > F_1$.) For the inductive step, assume that $\text{EUCLID}(F_k, F_{k-1})$ makes exactly $k - 2$ recursive calls. For $k > 2$, we have $F_k > F_{k-1} > 0$ and $F_{k+1} = F_k + F_{k-1}$, and so by Exercise 31.1-1, we

have $F_{k+1} \bmod F_k = F_{k-1}$. Because $\text{EUCLID}(a, b)$ calls $\text{EUCLID}(b, a \bmod b)$ when $b > 0$, the call $\text{EUCLID}(F_{k+1}, F_k)$ recurses one time more than the call $\text{EUCLID}(F_k, F_{k-1})$, or exactly $k - 1$ times, which meets the upper bound given by Theorem 31.11.

Since F_k is approximately $\phi^k / \sqrt{5}$, where ϕ is the golden ratio $(1 + \sqrt{5})/2$ defined by equation (3.32) on page 69, the number of recursive calls in EUCLID is $O(\lg b)$. (See Exercise 31.2-5 for a tighter bound.) Therefore, a call of EUCLID on two β -bit numbers performs $O(\beta)$ arithmetic operations and $O(\beta^3)$ bit operations (assuming that multiplication and division of β -bit numbers take $O(\beta^2)$ bit operations). Problem 31-2 asks you to prove an $O(\beta^2)$ bound on the number of bit operations.

The extended form of Euclid's algorithm

By rewriting Euclid's algorithm, we can gain additional useful information. Specifically, let's extend the algorithm to compute the integer coefficients x and y such that

$$d = \gcd(a, b) = ax + by, \quad (31.16)$$

where either or both of x and y may be zero or negative. These coefficients will prove useful later for computing modular multiplicative inverses. The procedure EXTENDED-EUCLID takes as input a pair of nonnegative integers and returns a triple of the form (d, x, y) that satisfies equation (31.16). As an example, Figure 31.1 traces out the call $\text{EXTENDED-EUCLID}(99, 78)$.

```

EXTENDED-EUCLID( $a, b$ )
1  if  $b == 0$ 
2      return ( $a, 1, 0$ )
3  else ( $d', x', y'$ ) = EXTENDED-EUCLID( $b, a \bmod b$ )
4      ( $d, x, y$ ) = ( $d', y', x' - \lfloor a/b \rfloor y'$ )
5      return ( $d, x, y$ )

```

The EXTENDED-EUCLID procedure is a variation of the EUCLID procedure. Line 1 is equivalent to the test " $b == 0$ " in line 1 of EUCLID . If $b = 0$, then EXTENDED-EUCLID returns not only $d = a$ in line 2, but also the coefficients $x = 1$ and $y = 0$, so that $a = ax + by$. If $b \neq 0$, EXTENDED-EUCLID first computes (d', x', y') such that $d' = \gcd(b, a \bmod b)$ and

$$d' = bx' + (a \bmod b)y'. \quad (31.17)$$

As in the EUCLID procedure, we have $d = \gcd(a, b) = d' = \gcd(b, a \bmod b)$. To obtain x and y such that $d = ax + by$, let's rewrite equation (31.17), setting

a	b	$\lfloor a/b \rfloor$	d	x	y
99	78	1	3	-11	14
78	21	3	3	3	-11
21	15	1	3	-2	3
15	6	2	3	1	-2
6	3	2	3	0	1
3	0	—	3	1	0

Figure 31.1 How EXTENDED-EUCLID computes $\gcd(99, 78)$. Each line shows one level of the recursion: the values of the inputs a and b , the computed value $\lfloor a/b \rfloor$, and the values d , x , and y returned. The triple (d, x, y) returned becomes the triple (d', x', y') used at the next higher level of recursion. The call EXTENDED-EUCLID(99, 78) returns $(3, -11, 14)$, so that $\gcd(99, 78) = 3 = 99 \cdot (-11) + 78 \cdot 14$.

$d = d'$ and using equation (3.11):

$$\begin{aligned} d &= bx' + (a - b \lfloor a/b \rfloor)y' \\ &= ay' + b(x' - \lfloor a/b \rfloor y'). \end{aligned}$$

Thus, choosing $x = y'$ and $y = x' - \lfloor a/b \rfloor y'$ satisfies the equation $d = ax + by$, thereby proving the correctness of EXTENDED-EUCLID.

Since the number of recursive calls made in EUCLID is equal to the number of recursive calls made in EXTENDED-EUCLID, the running times of EUCLID and EXTENDED-EUCLID are the same, to within a constant factor. That is, for $a > b > 0$, the number of recursive calls is $O(\lg b)$.

Exercises

31.2-1

Prove that equations (31.11) and (31.12) imply equation (31.13).

31.2-2

Compute the values (d, x, y) that the call EXTENDED-EUCLID(899, 493) returns.

31.2-3

Prove that for all integers a, k , and n ,

$$\gcd(a, n) = \gcd(a + kn, n). \quad (31.18)$$

Use equation (31.18) to show that $a = 1 \pmod{n}$ implies $\gcd(a, n) = 1$.

31.2-4

Rewrite EUCLID in an iterative form that uses only a constant amount of memory (that is, stores only a constant number of integer values).

31.2-5

If $a > b \geq 0$, show that the call $\text{EUCLID}(a, b)$ makes at most $1 + \log_\phi b$ recursive calls. Improve this bound to $1 + \log_\phi(b / \gcd(a, b))$.

31.2-6

What does $\text{EXTENDED-EUCLID}(F_{k+1}, F_k)$ return? Prove your answer correct.

31.2-7

Define the gcd function for more than two arguments by the recursive equation $\gcd(a_0, a_1, \dots, a_n) = \gcd(a_0, \gcd(a_1, a_2, \dots, a_n))$. Show that the gcd function returns the same answer independent of the order in which its arguments are specified. Also show how to find integers $x_0, x_1, \dots, x_n$ such that $\gcd(a_0, a_1, \dots, a_n) = a_0x_0 + a_1x_1 + \dots + a_nx_n$. Show that the number of divisions performed by your algorithm is $O(n + \lg(\max\{a_0, a_1, \dots, a_n\}))$.

31.2-8

The *least common multiple* $\text{lcm}(a_1, a_2, \dots, a_n)$ of integers $a_1, a_2, \dots, a_n$ is the smallest nonnegative integer that is a multiple of each a_i . Show how to compute $\text{lcm}(a_1, a_2, \dots, a_n)$ efficiently using the (two-argument) gcd operation as a sub-routine.

31.2-9

Prove that n_1, n_2, n_3 , and n_4 are pairwise relatively prime if and only if

$$\gcd(n_1n_2, n_3n_4) = \gcd(n_1n_3, n_2n_4) = 1.$$

More generally, show that $n_1, n_2, \dots, n_k$ are pairwise relatively prime if and only if a set of $\lceil \lg k \rceil$ pairs of numbers derived from the n_i are relatively prime.

31.3 Modular arithmetic

Informally, you can think of modular arithmetic as arithmetic as usual over the integers, except that when working modulo n , then every result x is replaced by the element of $\{0, 1, \dots, n-1\}$ that is equivalent to x , modulo n (so that x is replaced by $x \bmod n$). This informal model suffices if you stick to the operations of addition, subtraction, and multiplication. A more formal model for modular arithmetic, which follows, is best described within the framework of group theory.

Finite groups

A **group** $(S, \oplus)$ is a set S together with a binary operation $\oplus$ defined on S for which the following properties hold:

1. **Closure:** For all $a, b \in S$, we have $a \oplus b \in S$.
2. **Identity:** There exists an element $e \in S$, called the **identity** of the group, such that $e \oplus a = a \oplus e = a$ for all $a \in S$.
3. **Associativity:** For all $a, b, c \in S$, we have $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.
4. **Inverses:** For each $a \in S$, there exists a unique element $b \in S$, called the **inverse** of a , such that $a \oplus b = b \oplus a = e$.

As an example, consider the familiar group $(\mathbb{Z}, +)$ of the integers $\mathbb{Z}$ under the operation of addition: 0 is the identity, and the inverse of a is $-a$. An **abelian group** $(S, \oplus)$ satisfies the **commutative law** $a \oplus b = b \oplus a$ for all $a, b \in S$. The **size** of group $(S, \oplus)$ is $|S|$, and if $|S| < \infty$, then $(S, \oplus)$ is a **finite group**.

The groups defined by modular addition and multiplication

We can form two finite abelian groups by using addition and multiplication modulo n , where n is a positive integer. These groups are based on the equivalence classes of the integers modulo n , defined in Section 31.1.

To define a group on $\mathbb{Z}_n$, we need suitable binary operations, which we obtain by redefining the ordinary operations of addition and multiplication. We can define addition and multiplication operations for $\mathbb{Z}_n$, because the equivalence class of two integers uniquely determines the equivalence class of their sum or product. That is, if $a = a' \pmod{n}$ and $b = b' \pmod{n}$, then

$$\begin{aligned} a + b &= a' + b' \pmod{n}, \\ ab &= a'b' \pmod{n}. \end{aligned}$$

Thus, we define addition and multiplication modulo n , denoted $+_n$ and $\cdot_n$, by

$$\begin{aligned} [a]_n +_n [b]_n &= [a + b]_n, \\ [a]_n \cdot_n [b]_n &= [ab]_n. \end{aligned} \tag{31.19}$$

(We can define subtraction similarly on $\mathbb{Z}_n$ by $[a]_n -_n [b]_n = [a - b]_n$, but division is more complicated, as we'll see.) These facts justify the common and convenient practice of using the smallest nonnegative element of each equivalence class as its representative when performing computations in $\mathbb{Z}_n$. We add, subtract, and multiply as usual on the representatives, but we replace each result x by the representative of its class, that is, by $x \bmod n$.

$+_6$	0	1	2	3	4	5		$\cdot_{15}$	1	2	4	7	8	11	13	14
0	0	1	2	3	4	5		1	1	2	4	7	8	11	13	14
1	1	2	3	4	5	0		2	2	4	8	14	1	7	11	13
2	2	3	4	5	0	1		4	4	8	1	13	2	14	7	11
3	3	4	5	0	1	2		7	7	14	13	4	11	2	1	8
4	4	5	0	1	2	3		8	8	1	2	11	4	13	14	7
5	5	0	1	2	3	4		11	11	7	14	2	13	1	8	4
								13	13	11	7	1	14	8	4	2
								14	14	13	11	8	7	4	2	1

(a)
(b)

Figure 31.2 Two finite groups. Equivalence classes are denoted by their representative elements. **(a)** The group $(\mathbb{Z}_6, +_6)$. **(b)** The group $(\mathbb{Z}_{15}^*, \cdot_{15})$.

Using this definition of addition modulo n , we define the **additive group modulo n** as $(\mathbb{Z}_n, +_n)$. The size of the additive group modulo n is $|\mathbb{Z}_n| = n$. Figure 31.2(a) gives the operation table for the group $(\mathbb{Z}_6, +_6)$.

Theorem 31.12

The system $(\mathbb{Z}_n, +_n)$ is a finite abelian group.

Proof Equation (31.19) shows that $(\mathbb{Z}_n, +_n)$ is closed. Associativity and commutativity of $+_n$ follow from the associativity and commutativity of $+$:

$$\begin{aligned}
 ([a]_n +_n [b]_n) +_n [c]_n &= [a + b]_n +_n [c]_n \\
 &= [(a + b) + c]_n \\
 &= [a + (b + c)]_n \\
 &= [a]_n +_n [b + c]_n \\
 &= [a]_n +_n ([b]_n +_n [c]_n) ,
 \end{aligned}$$

$$\begin{aligned}
 [a]_n +_n [b]_n &= [a + b]_n \\
 &= [b + a]_n \\
 &= [b]_n +_n [a]_n .
 \end{aligned}$$

The identity element of $(\mathbb{Z}_n, +_n)$ is 0 (that is, $[0]_n$). The (additive) inverse of an element a (that is, of $[a]_n$) is the element $-a$ (that is, $[-a]_n$ or $[n - a]_n$), since $[a]_n +_n [-a]_n = [a - a]_n = [0]_n$. ■